

Data Directory -- OCP Maintenance AI MVP

This directory contains the authoritative equipment and component type libraries used by the OCP Maintenance AI system for phosphate processing plants (OCP Group, Morocco).

Files

`equipment_library.json`

Master library of **equipment types** found in phosphate beneficiation plants. Each equipment type defines:

- Equipment identification (ID, name in EN/FR/AR, category)
- Operational parameters (power, weight, annual hours, expected life)
- Criticality classification (AA through D)
- OEM manufacturers
- **Sub-assemblies** broken down into maintainable items, each with:
- Cross-reference to the component library (`component_lib_ref`)
- Failure modes with mechanism + cause (validated against the 72-combo table)
- Weibull parameters (beta, eta) for reliability modeling
- Maintenance strategy (condition-based, fixed-time, run-to-failure, fault-finding)
- Standard task templates with type, frequency, and constraint

Current equipment types: SAG Mill, Ball Mill, Rod Mill, Slurry Pump, Flotation Cell, Belt Conveyor, Thickener, Belt Filter, Rotary Dryer, Crusher, Vibrating Screen, Hydrocyclone, Agitator, Compressor, Heat Exchanger.

`component_library.json`

Master library of **standard industrial component types** used across multiple equipment types. Each component type defines:

- Component identification (ID, type name, category)
- Technical description specific to phosphate processing context
- OEM manufacturers and common model numbers
- Typical service life in hours
- **Generic failure modes** with mechanism + cause and Weibull parameters
- **Standard maintenance task templates** with task type and frequency

Component categories: Bearings, Seals, Motors, Gearboxes, Couplings, Impellers, Liners, Filters, Belts, Valves, Instruments.

Data Structure

Equipment Library Structure

```
equipment_library.json
+-- _meta                # Version, description, counts
+-- equipment_types[]    # Array of equipment type objects
  +-- equipment_type_id  # Unique ID (e.g., "ET-SAG-MILL")
  +-- name / name_fr / name_ar
  +-- category           # Equipment category enum
  +-- tag_convention     # Asset tag pattern
  +-- typical_power_kw   # Nominal power rating
  +-- criticality_class  # AA, A+, A, B, C, or D
  +-- manufacturers[]   # List of OEM names
  +-- sub_assemblies[]  # Hierarchical breakdown
    +-- name / name_fr
    +-- order           # Display order
  +-- maintainable_items[]
    +-- name / name_fr
    +-- component_lib_ref # Links to component_library.json
    +-- failure_modes[]
      +-- what           # Natural-language description
      +-- mechanism      # From Mechanism enum (18 values)
      +-- cause          # From Cause enum (validated pair)
      +-- failure_pattern # Weibull curve type (A-F)
      +-- failure_consequence
      +-- strategy_type
      +-- typical_task
      +-- task_type      # From TaskType enum
      +-- frequency_value / frequency_unit
      +-- constraint     # ONLINE / OFFLINE / TEST_MODE
      +-- weibull_beta   # Shape parameter (>0)
      +-- weibull_eta    # Characteristic life in days
```

Component Library Structure

```
component_library.json
+-- _meta                # Version, description, counts
+-- component_types[]    # Array of component type objects
  +-- component_type_id  # Unique ID (e.g., "CL-BEARING-SPHERICAL-ROLLER")
  +-- component_type     # Type key (e.g., "SPHERICAL_ROLLER_BEARING")
  +-- category           # Component category
  +-- description        # Technical description
  +-- manufacturers[]    # List of OEM names
  +-- common_models[]    # Common part numbers / model names
  +-- typical_life_hours # Expected service life (hours)
  +-- failure_modes[]
    +-- mechanism        # From Mechanism enum (18 values)
    +-- cause            # From Cause enum (validated pair)
    +-- weibull_beta     # Shape parameter (>0)
    +-- weibull_eta_hours # Characteristic life in hours (>0)
  +-- standard_tasks[]
    +-- task_type        # From TaskType enum
    +-- description      # Task description
    +-- frequency_weeks  # Recommended interval in weeks
```

Cross-Referencing

The `component_lib_ref` field in equipment library maintainable items links to the `component_type_id` in the component library. This allows:

1. **Reuse** -- The same component type (e.g., `CL-MOTOR-LV`) can appear in multiple equipment types (conveyor, pump, agitator).
2. **Defaults** -- When creating a new equipment instance, the component library provides default failure modes and tasks.
3. **Consistency** -- Standard Weibull parameters and task templates are maintained centrally.

How to Add a New Equipment Type

1. **Define the equipment type** in `equipment_library.json`: - Assign a unique `equipment_type_id` following the pattern `ET-{SHORT_NAME}`. - Provide names in all three languages (EN, FR, AR). - Set the criticality class (AA, A+, A, B, C, or D). - List at least one manufacturer.
2. **Break down into sub-assemblies**: - Each sub-assembly must have a `name`, `name_fr`, and `order`. - Each sub-assembly must contain at least one `maintainable_item`.
3. **Define failure modes for each maintainable item**: - Every failure mode MUST use a valid (mechanism, cause) pair from the 72-combo table. - Provide realistic Weibull beta (>0) and eta (>0) values. - Assign an appropriate task type from the TaskType enum.
4. **Cross-reference to component library**: - If the component type already exists in `component_library.json`, use its `component_type_id` as the `component_lib_ref`. - If not, first add the new component type to the component library.

5. **Validate**: - Run `pytest tests/test_equipment_library.py` to verify structural integrity. - Ensure all mechanism+cause pairs are valid (see validation section below).

How to Add a New Component Type

1. **Add to** `component_library.json` : - Assign a unique `component_type_id` following the pattern `CL-{CATEGORY}-{NAME}` . - Assign a unique `component_type` key (UPPER_SNAKE_CASE). - Include at least one failure mode and one standard task.
2. **Validate**: Run the test suite to verify.

Validation Against the 72-Combo Table

All failure modes must use mechanism+cause pairs from the authoritative 72-combination lookup table defined in `tools/models/schemas.py` as `VALID_FM_COMBINATIONS` . This table originates from source document SRC-09: "Failure Modes (Mechanism + Cause).xlsx".

Valid Mechanisms (18 total)

Mechanism	Description
ARCS	Electrical arcing damage
BLOCKS	Flow or movement obstruction
BREAKS_FRACTURE_SEPARATES	Mechanical fracture or separation
CORRODES	Corrosion degradation
CRACKS	Crack initiation or propagation
DEGRADES	General material degradation
DISTORTS	Permanent deformation
DRIFTS	Measurement or calibration drift
EXPIRES	End of useful life (time-limited items)
IMMOBILISED	Seized or stuck condition
LOOSES_PRELOAD	Loss of bolt or spring preload
OPEN_CIRCUIT	Electrical open circuit failure
OVERHEATS_MELTS	Thermal damage from overheating
SEVERS	Complete cutting or severing
SHORT_CIRCUITS	Electrical short circuit
THERMALLY_OVERLOADS	Thermal overload trip or damage
WASHES_OFF	Material removal by fluid action
WEARS	Progressive material loss from wear

Validation Rule

For any failure mode entry:

```
(mechanism, cause) in VALID_FM_COMBINATIONS # Must be True
```

To see all valid pairs, refer to [tools/models/schemas.py](#) or run:

```
from tools.models.schemas import VALID_FM_COMBINATIONS
for mech, cause in sorted(VALID_FM_COMBINATIONS):
    print(f" {mech.value} + {cause.value}")
```

Version History

Version	Date	Changes
1.0.0	2025-05-15	Initial release -- 15 equipment types, 30 component types